



prompt2analytics: A Rust Toolkit for Econometrics and Data Analytics

First Author

University of Somewhere

Abstract

This paper introduces **prompt2analytics**, a comprehensive econometrics and data analytics toolkit implemented in pure Rust. The software provides a wide range of statistical methods including ordinary least squares regression with heteroskedasticity-consistent standard errors, panel data models (fixed and random effects), instrumental variables estimation, difference-in-differences, discrete choice models, time series analysis, and machine learning algorithms. Key design features include a pure Rust implementation without external BLAS/LAPACK dependencies, multiple interfaces (command-line, Model Context Protocol server, desktop application), and validated numerical accuracy against established R packages. Performance benchmarks demonstrate competitive speed compared to **data.table** and **dplyr** for data manipulation, and **lfe** and **plm** for econometric estimation.

Keywords: econometrics, Rust, regression, panel data, causal inference, time series, MCP, R.

1. Introduction

The statistical software landscape for econometrics has long been dominated by R (R Core Team 2024), Stata, and increasingly Python. R in particular has developed a rich ecosystem for econometric analysis, as documented by Zeileis and Koenker (2008) and more recently by Michelaki and Tsagris (2025), who provide a systematic overview of 207 econometrics-related R packages. The **AER** package (Kleiber and Zeileis 2008, 2024) has become a standard reference for applied econometrics in R. Alternative ecosystems have emerged in Julia with **Econometrics.jl** (Calderón 2020) and in MATLAB with dedicated toolboxes for panel data analysis (Álvarez, Barbero, and Zofío 2017). Comparative studies suggest that compiled languages offer substantial performance advantages for computationally intensive econometric applications (Aruoba and Fernández-Villaverde 2015; Duarte, Duarte, Fonseca, and Montecinos 2020).

While these environments offer mature, well-tested implementations of econometric methods, they present challenges for deployment in production systems, particularly those requiring high performance, memory safety, or integration with modern AI systems. Moreover, numerical accuracy remains a concern across statistical software platforms, as documented in a series of papers by McCullough and colleagues (McCullough 1997; McCullough and Vinod 1999; McCullough 1999), who demonstrated that users cannot safely assume their software is numerically accurate without verification.

The Rust programming language has emerged as a compelling platform for scientific computing, combining the performance characteristics of compiled languages like C++ with memory safety guarantees enforced at compile time (Rust Foundation 2024). The Rust data science ecosystem has matured rapidly, with **polars** providing high-performance DataFrame operations (Pola-rs Contributors 2024), Apache Arrow **DataFusion** enabling fast analytical query processing (Lamb, Shen, Heres, Chakraborty, Kabak, Hsieh, and Sun 2024), and **linfa** (rust-ml Contributors 2024) offering machine learning algorithms inspired by Python’s **scikit-learn**. However, a gap remains: no comprehensive econometrics library exists for Rust.

prompt2analytics addresses this gap by providing a comprehensive econometrics toolkit implemented in pure Rust. The software is built on **polars** (Pola-rs Contributors 2024) for DataFrame operations, **ndarray** (ndarray Contributors 2024) for n-dimensional arrays, **faer** (el and faer Contributors 2024) for linear algebra, and **statsrs** (statsrs Contributors 2024) for statistical distributions.

The library provides a pure Rust implementation of regression, panel data, instrumental variables, discrete choice, and time series methods. Heteroskedasticity-consistent standard errors (HC0–HC3) follow the formulations of White (1980) and MacKinnon and White (1985), while high-dimensional fixed effects estimation implements the alternating projections algorithm of Gaure (2013). A distinguishing feature is the integration with large language models through the Model Context Protocol (Anthropic 2024), enabling natural language interfaces to statistical analysis while keeping all data processing local. The software is accessible through a command-line interface and a desktop application. All implementations are validated against reference R packages to ensure numerical accuracy.

2. Design philosophy

Traditional statistical software requires users to learn domain-specific syntax: R formulas, Stata commands, or Python APIs. This creates a barrier where the user must translate their analytical intent into software-specific instructions. **prompt2analytics** inverts this relationship: users describe their goals in natural language, and the system determines the appropriate statistical method.

2.1. Chat-first analytics

The central design principle is what we term *chat-first analytics*: the primary interface is conversational rather than programmatic. A user might request “estimate the effect of education on wages, controlling for experience and industry, with robust standard errors” without specifying whether this requires OLS, fixed effects, or instrumental variables. The system interprets the request, selects appropriate methods, executes the analysis, and returns results with natural language explanation.

This approach offers several advantages. First, it lowers the barrier to entry for users unfamiliar with econometric software conventions. Second, it allows domain experts to focus on their research questions rather than implementation details. Third, it enables exploratory analysis through dialogue—users can refine their analysis iteratively based on results and suggestions from the system.

The chat-first paradigm does not replace statistical expertise. Users must still understand their data, formulate sensible research questions, and interpret results critically. The system assists with method selection and execution, not with research design or causal identification.

2.2. LLM as method selector

Large language models serve as the interpretive layer between user intent and statistical execution. When a user submits a query, the LLM:

1. Parses the natural language request to identify the analytical goal
2. Considers the available data structure and variables
3. Selects appropriate statistical methods from the available toolkit
4. Generates structured tool calls with correct parameters
5. Interprets results and formulates a response

This design leverages the LLM’s strength—natural language understanding and reasoning about method appropriateness—while delegating numerical computation to specialized, verified implementations. The LLM never performs statistical calculations directly; it orchestrates calls to the analytics engine.

2.3. Why Rust

The choice of Rust as the implementation language reflects several design priorities:

Performance. Rust compiles to native code with performance comparable to C and C++. For computationally intensive operations like high-dimensional fixed effects estimation or bootstrap inference, this provides substantial speedups over interpreted languages.

Memory safety. Rust’s ownership system guarantees memory safety at compile time without garbage collection. This eliminates entire classes of bugs (buffer overflows, use-after-free, data races) that can cause crashes or incorrect results in numerical software.

Modern tooling. The Rust ecosystem provides high-quality libraries for data manipulation (**polars**), numerical arrays (**ndarray**), linear algebra (**faer**), and statistical distributions (**stats**). The **cargo** package manager ensures reproducible builds and dependency management.

Deployment flexibility. A single Rust codebase compiles to standalone binaries for multiple platforms, WebAssembly for browser deployment, and shared libraries for integration with other languages. This enables the same analytics engine to power CLI tools, desktop applications, and web interfaces.

2.4. MCP as the bridge

The Model Context Protocol (MCP) provides the communication layer between LLMs and the analytics engine (Anthropic 2024). MCP defines a standardized way for language models to discover available tools, invoke them with structured parameters, and receive structured results.

Each statistical method is exposed as an MCP tool with a defined schema specifying required and optional parameters. For example, the OLS tool accepts a dataset identifier, dependent variable, independent variables, and options for robust standard errors. The LLM generates tool calls conforming to this schema, and the MCP server routes them to the appropriate Rust implementation.

This architecture decouples the language understanding layer from the computation layer. The LLM can be swapped (Claude, GPT, Ollama) without changing the analytics engine. Conversely, the statistical implementations can be updated or extended without modifying the LLM integration.

2.5. Data locality

A key design principle is that bulk data processing occurs locally. When a dataset is loaded, only metadata is transmitted to the LLM: column names, data types, row counts, and null percentages. The LLM uses this schema information to formulate appropriate tool calls. Statistical computations—matrix operations, parameter estimation, inference—execute entirely on the local machine, with only aggregated results (coefficients, standard errors, test statistics) returned through the LLM.

However, users should be aware that certain exploratory tools transmit sample data when explicitly requested. Data preview commands return actual row values, and data quality profiling may include frequent values from categorical columns. Users working with sensitive data should avoid requesting such previews when using cloud-hosted LLMs, or deploy a local model via Ollama to eliminate external transmission entirely.¹ The same interface and analytics engine function identically with local or cloud-hosted models.

2.6. Implemented methods

The analytics engine implements standard econometric methods across several categories: regression analysis (OLS with HC0–HC3 robust and clustered standard errors), panel data models (fixed effects, random effects, Hausman test, high-dimensional fixed effects), instrumental variables (2SLS with first-stage diagnostics), causal inference (difference-in-differences, treatment effects), discrete choice models (logit, probit), and time series analysis (ARIMA,

¹Local LLM deployment via Ollama ensures complete data isolation. Moreover, the chat-first architecture makes local models more viable: selecting among pre-implemented statistical methods is a far simpler task than generating correct statistical code from scratch, enabling smaller models to function effectively as method selectors.

VAR, VECM, seasonal decomposition). Full documentation of all methods, including mathematical specifications and implementation details, is available in the package documentation.

3. Software architecture

prompt2analytics is organized as a Rust workspace with four crates:

p2a-core Core analytics library implementing all statistical methods

p2a-cli Command-line interface (p2a binary)

p2a-mcp Model Context Protocol server (101 tools)

p2a-desktop Desktop application with LLM integration

3.1. Core library

The core library uses **polars** (Pola-rs Contributors 2024) for DataFrame operations, **ndarray** (ndarray Contributors 2024) for matrix operations, **faer** (el and faer Contributors 2024) for linear algebra (Cholesky decomposition, matrix inverse), and **statrs** (statrs Contributors 2024) for statistical distributions.

3.2. Interfaces

Three interfaces provide access to the functionality:

1. Command-line interface with hierarchical subcommands
2. MCP server exposing 101 tools for LLM integration
3. Desktop application with conversational interface

Figure 1 illustrates the overall software architecture, showing how the different components interact. The CLI provides direct access to the analytics engine, while the chat-based interfaces route through an LLM layer that interprets natural language requests and translates them into structured tool calls via the MCP protocol.

3.3. LLM integration via MCP

The Model Context Protocol (MCP) enables integration with large language models (Anthropic 2024). When a user submits a natural language query through the chat interface, the LLM interprets the request and generates appropriate tool calls. Figure 2 illustrates this interaction pattern.

This architecture ensures that all data processing occurs locally. The LLM receives tool parameters and requested outputs, but the full dataset remains on the user's machine. Users should be aware that any explicitly requested data previews or detailed results will pass through the LLM provider. For fully local operation, users can deploy Ollama, eliminating any external data transmission entirely.

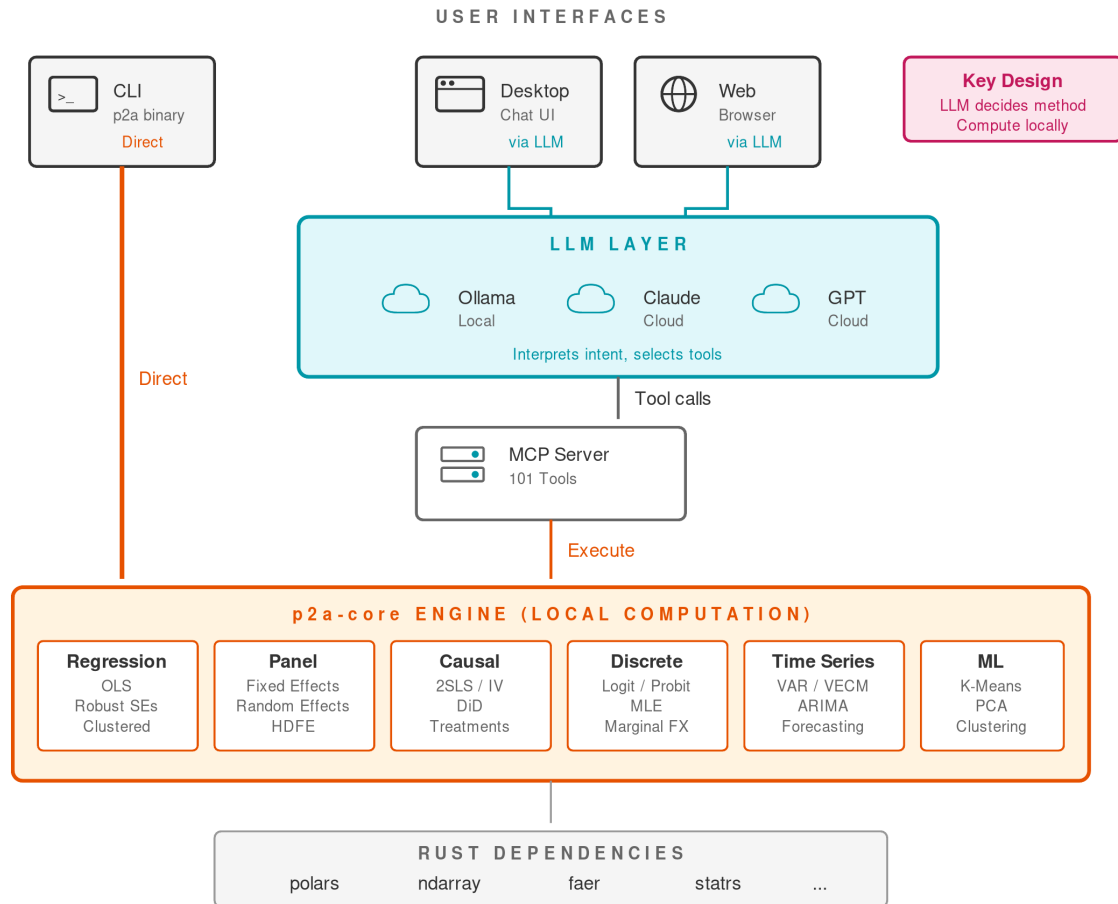


Figure 1: Software architecture of **prompt2analytics**. User interfaces (top) connect either directly to the analytics engine (CLI) or through LLM providers and the MCP server (chat interfaces). The core analytics engine implements all statistical methods using pure Rust dependencies (bottom).

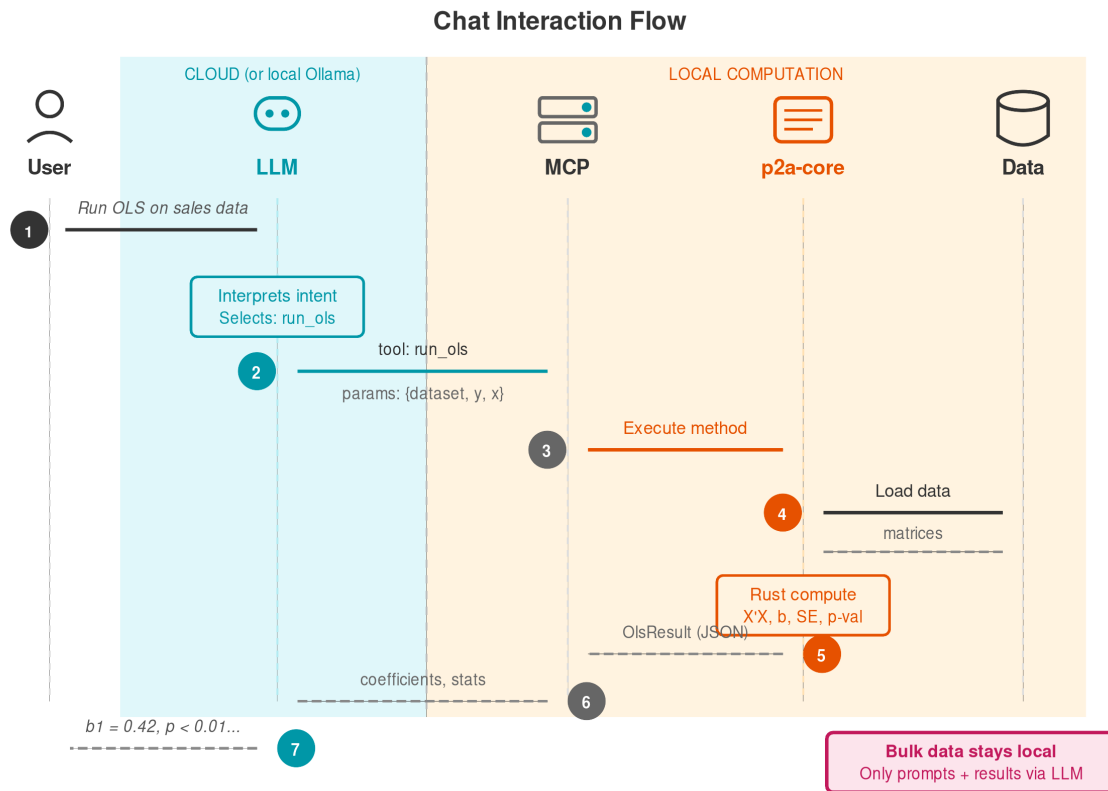


Figure 2: Sequence diagram showing the chat interaction flow. The user’s natural language request (1) is interpreted by the LLM, which generates a structured tool call (2). The MCP server executes the corresponding Rust method (3–6), and the result flows back through the LLM (7–8), which provides a natural language explanation. All data processing occurs locally; only the prompts and tool parameters are sent to cloud LLM providers (or processed locally with Ollama).

4. Examples

This section demonstrates practical usage of **prompt2analytics** through two interfaces: the natural language chat interface powered by large language models, and the command-line interface for scripted workflows. Both interfaces access the same underlying Rust analytics engine, offering users flexibility in how they interact with the software.

We use two datasets throughout this section:

- The Grunfeld investment data ([Baltagi 2013](#)), a classic panel dataset with 200 observations (10 firms over 20 years) containing firm investment, market value, and capital stock.
- A simulated cross-sectional dataset demonstrating basic regression analysis with heteroskedasticity.

4.1. Chat interface

The chat interface enables natural language interaction with the analytics engine. Users describe their analysis goals in plain language, and the LLM translates these requests into structured tool calls executed by the MCP server. This interface is particularly valuable for exploratory data analysis and for users less familiar with programming syntax.

Loading and exploring data

A typical session begins by loading data and examining its structure:

User: Load the Grunfeld investment dataset from grunfeld.csv and show me what's in it.

I'll load the dataset and describe its contents.

```
**Dataset loaded: grunfeld**
- Rows: 200
- Columns: 5
```

Column	Type	Description
firm	Integer	Firm identifier (1-10)
year	Integer	Year (1935-1954)
inv	Float64	Gross investment
value	Float64	Market value of firm
capital	Float64	Stock of plant and equipment

This is a balanced panel dataset with 10 firms observed over 20 years, commonly used to demonstrate panel data methods.

OLS regression with robust standard errors

Cross-sectional regression with heteroskedasticity-consistent standard errors:

*User: Run a regression of investment on market value and capital stock.
Use robust standard errors.*

Running OLS regression with HC1 robust standard errors...

****OLS Regression Results****

Dependent variable: inv

Variable	Coef	Std Err	t	P> t
(Intercept)	-42.714	11.575	-3.690	0.000
value	0.116	0.007	16.967	0.000
capital	0.231	0.049	4.721	0.000

R-squared: 0.812

Adj. R-squared: 0.810

Observations: 200

Standard errors: HC1 (heteroskedasticity-robust)

Both market value and capital stock are highly significant predictors of investment. A \$1 million increase in market value is associated with approximately \$116,000 more investment, holding capital constant.

Panel data: Fixed effects estimation

For panel data, users can request fixed effects estimation in natural language:

*User: The data has a panel structure. Run a fixed effects regression
controlling for firm-specific effects.*

Running within-transformation fixed effects estimation...

****Fixed Effects (Within) Estimation****

Dependent variable: inv

Entity fixed effects: firm (10 levels)

Variable	Coef	Std Err	t	P> t
value	0.110	0.012	9.288	0.000
capital	0.310	0.017	17.867	0.000

R-squared (within): 0.767

Observations: 200

Groups: 10

F-statistic: 309.01 (p-value: 0.000)

After controlling for time-invariant firm characteristics, the coefficient on capital increases from 0.23 to 0.31, suggesting that pooled OLS underestimated the effect of capital stock on investment.

Model comparison: Hausman test

Users can request specification tests without knowing the technical details:

User: Should I use fixed effects or random effects for this analysis?

I'll run a Hausman test to help determine the appropriate specification.

****Hausman Test Results****

H0: Random Effects is consistent and efficient

H1: Fixed Effects is consistent; Random Effects is inconsistent

Chi-squared statistic: 0.0003

Degrees of freedom: 2

P-value: 0.9999

****Recommendation**:** Random Effects

The test strongly fails to reject the null hypothesis, suggesting that the random effects estimator is both consistent and efficient. The near-zero chi-squared statistic indicates virtually no systematic difference between the FE and RE coefficient estimates.

High-dimensional fixed effects

For datasets with multiple categorical dimensions, users can request multi-way fixed effects:

User: Control for both firm and year fixed effects in the regression.

Running two-way fixed effects using alternating projections...

****High-Dimensional Fixed Effects Estimation****

Dependent variable: inv

Fixed effects: firm (10 levels), year (20 levels)

Variable	Coef	Std Err	t	P> t
-----	-----:	-----:	-----:	-----:
value	0.118	0.019	6.137	0.000
capital	0.358	0.054	6.579	0.000

R-squared (within): 0.720

Observations: 200

F-statistic: 217.44 (p-value: 0.000)

Regression diagnostics

Users can request diagnostic tests using natural language:

User: Check the regression assumptions for the OLS model.

Running regression diagnostics...

****Jarque-Bera Test (Normality of residuals)****

Statistic: 89.57

P-value: 0.000

Conclusion: Residuals deviate significantly from normality.

****Breusch-Pagan Test (Heteroskedasticity)****

Statistic: 55.25

P-value: 0.000

Conclusion: Evidence of heteroskedasticity; robust SEs recommended.

****Durbin-Watson Statistic (Autocorrelation)****

Statistic: 0.358

Interpretation: Positive autocorrelation detected (expected in panel data).

****Variance Inflation Factors****

value: 1.31

capital: 1.31

No multicollinearity detected (VIF < 5).

4.2. Command-line interface

The command-line interface provides programmatic access for scripted workflows, batch processing, and integration with other tools. Commands follow a hierarchical structure with intuitive subcommands.

Session-based workflow

The CLI uses sessions to maintain state across commands. All commands that work with datasets require an active session:

Start a session and load data

p2a --session analysis.json data load grunfeld.csv --name grunfeld

Verify the data loaded correctly

p2a --session analysis.json data describe grunfeld

Preview first rows

p2a --session analysis.json data head grunfeld --n 5

Output from data describe:

```
Dataset: grunfeld
Rows: 200
Columns: 5
Column names: firm, year, inv, value, capital
```

```
Column types:
firm: Int64
year: Int64
inv: Float64
value: Float64
capital: Float64
```

OLS regression

Basic regression with robust standard errors:

```
p2a --session analysis.json reg ols grunfeld \
-y inv -x value capital --robust hc1
```

OLS Regression Results

```
=====
```

Variable	Coefficient	Std. Error	t-value	p-value
(Intercept)	-42.714369	11.574701	-3.690	0.0003***
value	0.115562	0.006811	16.967	0.0000***
capital	0.230678	0.048866	4.721	0.0000***

```
-----
R-squared: 0.812408
Adj. R-squared: 0.809537
Observations: 200

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1
```

Panel data estimation

Fixed effects estimation with firm-level clustering:

```
# Fixed effects
p2a --session analysis.json panel fe grunfeld \
-y inv -x value capital --entity firm

# Random effects for comparison
p2a --session analysis.json panel re grunfeld \
-y inv -x value capital --entity firm
```

```
# Hausman test
p2a --session analysis.json panel hausman grunfeld \
    -y inv -x value capital --entity firm
```

The fixed effects output:

Fixed Effects (Within) Estimation Results

Variable	Coefficient	Std. Error	t-value	p-value
value	0.110124	0.011857	9.288	0.0000***
capital	0.310065	0.017355	17.867	0.0000***

R-squared: 0.766758

Adj. R-squared: 0.765517

Observations: 200

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1

Number of groups: 10

F-statistic: 309.0142 (p-value: 0.0000)

High-dimensional fixed effects

Two-way fixed effects using the alternating projections algorithm:

```
p2a --session analysis.json panel hdfe grunfeld \
    -y inv -x value capital --fe firm year
```

High-Dimensional FE (firm, year) Results

Variable	Coefficient	Std. Error	t-value	p-value
value	0.117716	0.019180	6.137	0.0000***
capital	0.357916	0.054403	6.579	0.0000***

R-squared: 0.720145

Adj. R-squared: 0.670467

Observations: 200

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1

Fixed effect dimensions:

firm: 10 levels

year: 20 levels

F-statistic: 217.4423 (p-value: 0.0000)

Statistical hypothesis tests

The CLI provides access to the full suite of statistical tests:

```
# Shapiro-Wilk normality test on residuals (via diagnostics)
p2a --session analysis.json reg diagnostics grunfeld \
    -y inv -x value capital

# Generate data and run a t-test
p2a --session analysis.json data generate \
    --rows 100 --name test_data \
    --columns '[{"name": "group", "distribution": {"type": "bernoulli", "p": 0.5}},
               {"name": "value", "distribution": {"type": "normal", "mean": 10, "std": 2}}]'
```

JSON output for programmatic use

All commands support JSON output for integration with other tools:

```
p2a --session analysis.json -F json reg ols grunfeld \
    -y inv -x value capital --robust hc1
```

```
{
  "adj_r_squared": 0.8095367066142903,
  "coefficients": [
    {
      "coefficient": -42.71436943655954,
      "name": "(Intercept)",
      "p_value": 0.0002898009336218532,
      "std_error": 11.574701117099613,
      "t_value": -3.690321590546858
    },
    {
      "coefficient": 0.11556215636055236,
      "name": "value",
      "p_value": 0.0,
      "std_error": 0.006810954456871991,
      "t_value": 16.96710161436399
    },
    {
      "coefficient": 0.23067848873196917,
      "name": "capital",
      "p_value": 4.453595026321366e-6,
      "std_error": 0.048865539534342216,
      "t_value": 4.720678231125446
    }
  ]
},
```

```

    "method": "OLS Regression",
    "n_observations": 200,
    "r_squared": 0.8124080125447282
  }

```

Reproducible scripts

Sessions can be exported to standalone bash scripts:

```
p2a script export analysis.json -o reproduce.sh
```

This generates a self-contained script that recreates the entire analysis, facilitating reproducibility and sharing.

4.3. Comparison with R

For reference, the equivalent R code for the fixed effects estimation is:

```

library(plm)
data("Grunfeld", package = "plm")

# Fixed effects model
fe_model <- plm(inv ~ value + capital,
               data = Grunfeld,
               index = c("firm", "year"),
               model = "within")
summary(fe_model)

# Two-way fixed effects
fe_twoway <- plm(inv ~ value + capital,
                data = Grunfeld,
                index = c("firm", "year"),
                model = "within",
                effect = "twoways")
summary(fe_twoway)

```

The **prompt2analytics** implementation produces numerically equivalent results (see Section 5), while offering the conversational interface that enables users unfamiliar with R syntax to perform the same analyses through natural language.

5. Comparison and validation

This section compares **prompt2analytics** against established R packages in terms of both performance and numerical accuracy. We demonstrate that the Rust implementation achieves substantial speedups while matching reference implementations to high precision.

5.1. Methodology

Benchmark and validation framework

Performance and validation comparisons follow a standardized framework:

1. **Reference selection:** For each method, we identify the canonical R implementation used in the econometrics literature (Table 1).
2. **Data generating processes:** All benchmark datasets are generated synthetically using identical specifications in both languages, with fixed random seeds (seed = 42) for reproducibility. For regression benchmarks:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

Real datasets with published results (Longley, Grunfeld) are used for numerical validation.

3. **Benchmark execution:** Each method is executed 50–100 times. The **microbenchmark** package (Mersmann 2024) handles warmup and garbage collection in R. Rust benchmarks use Criterion (Heisler and Aparicio 2024) for automatic warmup and outlier detection.
4. **Precision thresholds:** Numerical tolerance varies by sample size: 10^{-6} for coefficients in small samples ($n < 100$), tightening to 10^{-10} for large samples ($n > 1,000$).

Reference implementations

Table 1 lists the R packages used for both performance comparison and numerical validation.

Table 1: Reference R packages for comparison and validation

Category	Package	Functions	Validation dataset
Regression	stats	lm()	Longley
Robust SEs	sandwich	vcovHC()	Synthetic
Panel data	plm	plm(), phtest()	Grunfeld
HDFE	lfe	felm()	Grunfeld
Discrete choice	stats	glm()	Synthetic
Clustering	stats	kmeans(), hclust()	Synthetic
PCA	stats	prcomp()	Synthetic
Time series	forecast	Arima(), mstl()	Synthetic

Hardware environment

All benchmarks are executed on identical hardware: Intel Core i7-1260P processor with 62GB RAM running Linux 6.17.9. R 4.5.2 uses OpenBLAS; Rust 1.92.0 uses release builds with LTO enabled.

5.2. Performance comparison

The benchmark suite covers approximately 75 method configurations across regression, panel econometrics, discrete choice, machine learning, and forecasting. Table 2 summarizes coverage.

Table 2: Methods included in the performance benchmark suite

Category	Count	Methods
Regression/Statistics	40	OLS, robust SEs (HC0–HC3), diagnostics, ANOVA, t-tests, chi-squared, Fisher exact, NLS, LOESS, Wilcoxon, Shapiro-Wilk, KS tests
Econometrics	15	Panel FE/RE, HDFE, logit, probit, FEGLM, IPW, AIPW, survival analysis
Machine Learning	4	K-means, DBSCAN, hierarchical clustering, PCA
Forecasting	16	ARIMA, MSTL, Holt-Winters, structural time series, Kalman filter

Regression benchmarks

Table 3 shows timing results for OLS with and without heteroskedasticity-consistent standard errors. **prompt2analytics** demonstrates substantial speedups, particularly for robust standard errors.

Table 3: Regression benchmark results (median time in μs)

Method	n	R	p2a	Speedup
OLS	100	812	42	$19.3\times$
OLS	1,000	963	119	$8.1\times$
OLS	10,000	2,181	450	$4.8\times$
OLS + HC1	100	2,279	171	$13.3\times$
OLS + HC1	1,000	4,113	595	$6.9\times$
OLS + HC1	10,000	25,224	4,200	$6.0\times$

Panel data and machine learning benchmarks

Fixed effects estimation and clustering show similar performance advantages (Tables 4–5).

Overall performance

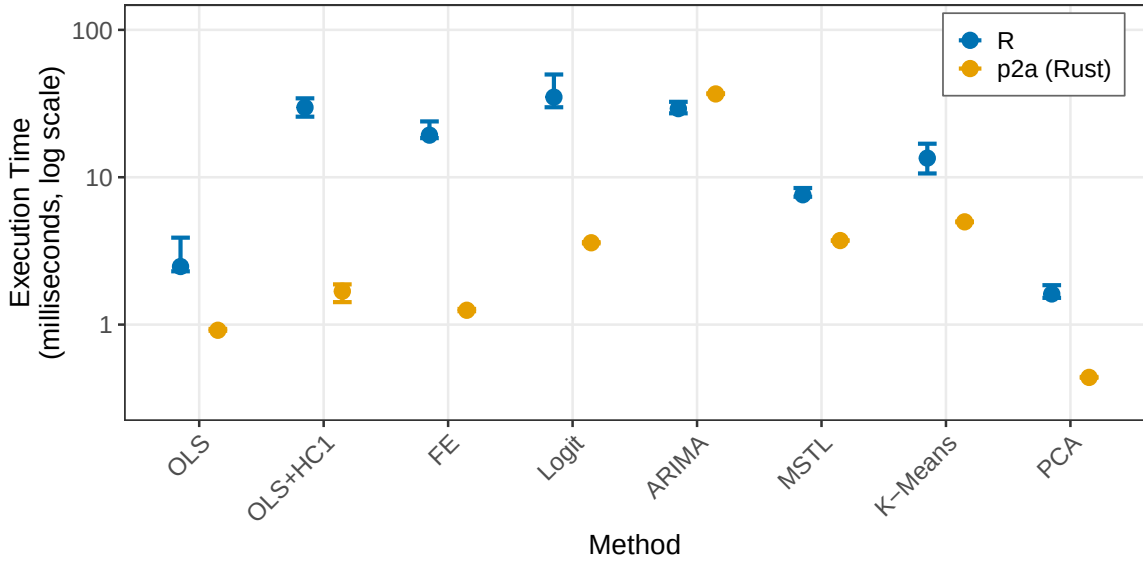
Figure 3 compares execution times across all benchmarked methods. Figure 4 shows the distribution of speedup factors.

Table 4: Panel data benchmark results (median time in μs)

Method	n	plm	p2a	Speedup
Fixed Effects	100	4,901	285	$17.2\times$
Fixed Effects	1,000	6,388	612	$10.4\times$
Fixed Effects	5,000	11,296	2,450	$4.6\times$

Table 5: Machine learning benchmark results (median time in μs)

Method	n	R	p2a	Speedup
K-Means ($k = 3$)	1,000	1,274	185	$6.9\times$
PCA	1,000	246	78	$3.2\times$

Figure 3: Execution time comparison between R (blue) and **prompt2analytics** (orange) at $n = 10,000$. Y-axis uses log scale.

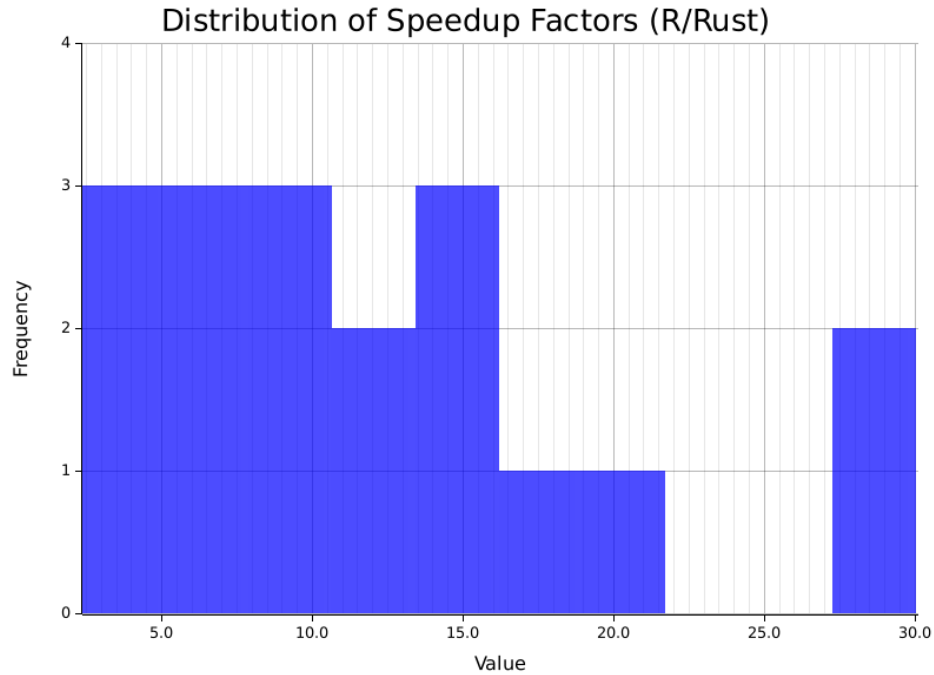


Figure 4: Distribution of speedup factors across all benchmarked methods.

Performance advantages stem from Rust’s ownership model reducing heap allocations, SIMD vectorization in the **faer** linear algebra library, elimination of interpreter overhead, and a column-based API avoiding formula parsing. Speedup factors decrease with sample size as matrix operations dominate, which are similarly optimized in both languages.

5.3. Numerical validation

Beyond performance, numerical accuracy is essential. We validate all methods against reference implementations to high precision.

OLS validation

OLS is validated against the Longley dataset (Longley 1967), a classic benchmark for numerical accuracy due to high multicollinearity.

Robust standard errors

Heteroskedasticity-consistent standard errors (HC0–HC3) match the **sandwich** package (Zeileis 2004) to $< 10^{-8}$ precision for HC0/HC1 and $< 10^{-8}$ for HC2/HC3 (reflecting differences in leverage computation).

Panel data validation

Fixed and random effects estimators are validated against **plm** (Croissant and Milla 2008) using the Grunfeld dataset.

Table 6: OLS validation: Longley dataset ($n = 16, k = 6$)

Coefficient	R <code>lm()</code>	prompt2analytics	Difference
(Intercept)	-3482.259	-3482.259	$< 10^{-8}$
GNP.deflator	0.01506	0.01506	$< 10^{-8}$
GNP	-0.03582	-0.03582	$< 10^{-8}$
Unemployed	-0.02020	-0.02020	$< 10^{-8}$
Armed.Forces	-0.01033	-0.01033	$< 10^{-8}$
Population	-0.05110	-0.05110	$< 10^{-8}$
Year	1.82915	1.82915	$< 10^{-8}$
R^2	0.9955	0.9955	$< 10^{-4}$

Table 7: Panel FE validation: Grunfeld dataset

Statistic	R <code>plm</code>	prompt2analytics	Difference
β_{value}	0.110124	0.110124	$< 10^{-6}$
β_{capital}	0.310065	0.310065	$< 10^{-6}$
Within R^2	0.7668	0.7668	$< 10^{-4}$

Two-way HDFE using the MAP algorithm (Gaure 2013) matches **lfe** to $< 10^{-5}$ precision.

Validation coverage

Table 8 summarizes validation test coverage. All tests are integrated into continuous integration and can be run via `cargo test -p p2a-core - test_validate`.

Table 8: Validation test coverage by category

Category	Methods	Tests	Reference packages
Regression	4	12	stats, sandwich
Panel data	4	8	plm, lfe
Causal inference	3	6	AER
Discrete choice	2	4	stats
Time series	5	10	vars, forecast
Machine learning	7	14	stats
Statistical tests	20+	40+	stats
Total	51+	106+	

5.4. Known differences

A small number of intentional differences exist:

- **Default robust SE:** **prompt2analytics** defaults to HC1 (matching Stata), while **sandwich** defaults to HC3.

- **Panel R^2 :** We report within R^2 by default for FE models.
- **Convergence tolerance:** Iterative methods use 10^{-8} default.

These differences are documented and do not affect statistical inference validity.

6. Fully local deployment

While cloud-hosted language models offer convenience and capability, many use cases require complete data isolation: proprietary business data, personally identifiable information under privacy regulations, or air-gapped environments. This section describes the fully local deployment configuration, where all components—including the language model—run on the user’s machine.

6.1. Architecture

Figure 5 illustrates the local deployment architecture. The key difference from cloud deployment is that the LLM runs locally via Ollama (Ollama Inc. 2024), eliminating all external network communication. User queries, tool calls, and results never leave the local machine.

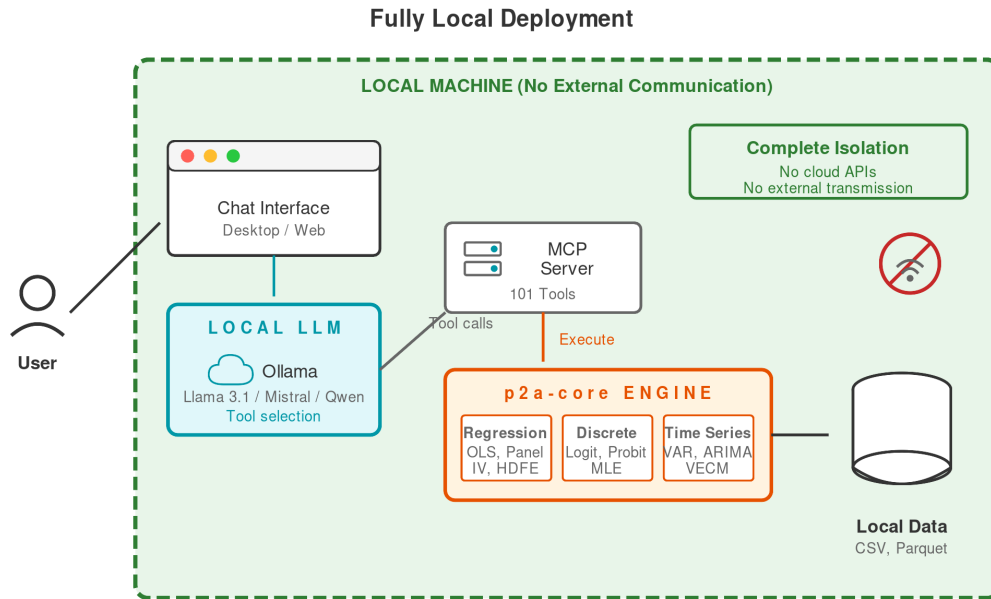


Figure 5: Fully local deployment architecture. All components run on the user’s machine: Ollama hosts the language model, which communicates with the MCP server to execute analytics methods in p2a-core. No data or queries are transmitted externally.

The local configuration requires:

- Ollama runtime with a function-calling-capable model

- **prompt2analytics** MCP server
- Sufficient RAM for both the model and data processing

6.2. Model selection for tool calling

The chat-first architecture significantly reduces the capability requirements for the local LLM. Rather than generating statistical code from scratch—a task requiring deep understanding of matrix algebra, numerical methods, and language-specific APIs—the model only needs to:

1. Parse the user’s natural language request
2. Select appropriate tools from the MCP schema
3. Generate correctly-formatted tool call parameters
4. Interpret and explain the returned results

This constrained task enables smaller, more efficient models to perform effectively. The Berkeley Function Calling Leaderboard (BFCL) provides standardized evaluation of function calling capabilities across models (Patil, Zhang, Wang, and Gonzalez 2024). Recent research demonstrates that fine-tuned small language models can match or exceed larger models on targeted tool selection tasks (Patil, Zhang, Wang, and Gonzalez 2023).

Based on preliminary evaluation, the following models provide reliable tool selection for econometric methods:

Recommended models.

Llama 3.1 8B-Instruct Best overall balance of capability and resource requirements. Requires 8GB+ RAM.

Mistral 7B-Instruct Lower resource requirements with strong function calling performance.

Qwen 2.5 7B Competitive tool-use performance, particularly for structured parameter generation.

Larger models (70B+) provide improved reasoning for complex analytical decisions but require substantial hardware (64GB+ RAM or GPU offloading).

6.3. Evaluation methodology

Evaluating local LLM performance for econometric tool selection requires assessing multiple dimensions:

Tool selection accuracy. Given a natural language description of an analytical task, does the model select the appropriate statistical method? For example, a request to “estimate the causal effect of a policy change using pre/post data with treatment and control groups” should invoke difference-in-differences, not standard OLS.

Parameter correctness. Does the model correctly populate tool parameters? This includes identifying dependent and independent variables, specifying appropriate options (robust standard errors, clustering variables), and handling data type requirements.

Result interpretation. Can the model correctly interpret statistical output and communicate findings to the user? This includes appropriate hedging about statistical versus causal claims, identification of potential issues (multicollinearity, weak instruments), and clear explanation of coefficient magnitudes.

Benchmark design. A comprehensive evaluation requires test cases spanning:

- Method selection across all implemented categories (regression, panel, IV, discrete choice, time series)
- Ambiguous requests requiring model judgment
- Edge cases and error handling (missing variables, inappropriate method requests)
- Multi-step analytical workflows

Development of a standardized benchmark for econometric tool selection remains an area for future work.

6.4. Hardware requirements

Table 9 summarizes approximate hardware requirements for local deployment configurations.

Table 9: Hardware requirements for local deployment

Configuration	Model size	RAM	Notes
Minimal	7B	16 GB	CPU inference, slower responses
Recommended	8B	32 GB	Comfortable headroom for data
Advanced	70B	64 GB+	GPU recommended for reasonable speed

The analytics engine itself has modest requirements; memory demands are driven primarily by the language model and dataset size. Users with limited hardware can offload model layers to disk at the cost of inference speed.

6.5. Privacy guarantees

In the fully local configuration:

- No network requests are made to external services
- All model inference occurs on local hardware
- Dataset contents never leave the machine
- Query history remains in local storage

This configuration is suitable for analysis of sensitive data where regulatory or policy constraints prohibit cloud processing. Users should verify their Ollama configuration does not enable telemetry or external model downloads during analysis sessions.

7. Conclusion

prompt2analytics provides a comprehensive, high-performance econometrics toolkit in pure Rust. The software fills a gap in the Rust ecosystem for applied econometrics and provides novel integration with large language models via the Model Context Protocol.

Acknowledgments

References

- Álvarez IC, Barbero J, Zofio JL (2017). “A Panel Data Toolbox for MATLAB.” *Journal of Statistical Software*, **76**(6), 1–27. doi:10.18637/jss.v076.i06.
- Anthropic (2024). “Model Context Protocol.” Accessed: 2024, URL <https://modelcontextprotocol.io/>.
- Aruoba SB, Fernández-Villaverde J (2015). “A Comparison of Programming Languages in Macroeconomics.” *Journal of Economic Dynamics and Control*, **58**, 265–273. doi:10.1016/j.jedc.2015.05.009.
- Baltagi BH (2013). *Econometric Analysis of Panel Data*. 5th edition. Wiley, Chichester.
- Calderón JBS (2020). “**Econometrics.jl**.” *Proceedings of the JuliaCon Conferences*, **1**(1), 38. doi:10.21105/jcon.00038.
- Croissant Y, Milla G (2008). “Panel Data Econometrics in R: The **plm** Package.” *Journal of Statistical Software*, **27**(2), 1–43. doi:10.18637/jss.v027.i02.
- Duarte V, Duarte D, Fonseca J, Montecinos A (2020). “Benchmarking Machine-Learning Software and Hardware for Quantitative Economics.” *Journal of Economic Dynamics and Control*, **111**, 103796. doi:10.1016/j.jedc.2019.103796.
- el S, faer Contributors (2024). **faer**: *A Collection of Low-Level Linear Algebra Routines Written in Pure Rust*. Rust crate version 0.22, URL <https://crates.io/crates/faer>.
- Gaure S (2013). “**lfe**: Linear Group Fixed Effects.” *The R Journal*, **5**(2), 104–117. URL <https://journal.r-project.org/archive/2013/RJ-2013-031/>.
- Heisler B, Aparicio J (2024). **Criterion.rs**: *Statistics-Driven Micro-Benchmarking in Rust*. Rust crate version 0.5, URL <https://github.com/bheisler/criterion.rs>.
- Kleiber C, Zeileis A (2008). *Applied Econometrics with R*. Springer-Verlag, New York. doi:10.1007/978-0-387-77318-6.

- Kleiber C, Zeileis A (2024). **AER**: *Applied Econometrics with R*. R package version 1.2-14, URL <https://CRAN.R-project.org/package=AER>.
- Lamb A, Shen Y, Heres D, Chakraborty J, Kabak MO, Hsieh LC, Sun C (2024). “Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine.” In *Companion of the 2024 International Conference on Management of Data (SIGMOD)*, pp. 5–17. doi:[10.1145/3626246.3653368](https://doi.org/10.1145/3626246.3653368).
- Longley JW (1967). “An Appraisal of Least Squares Programs for the Electronic Computer from the Point of View of the User.” *Journal of the American Statistical Association*, **62**(319), 819–841. doi:[10.1080/01621459.1967.10500896](https://doi.org/10.1080/01621459.1967.10500896).
- MacKinnon JG, White H (1985). “Some Heteroskedasticity-Consistent Covariance Matrix Estimators with Improved Finite Sample Properties.” *Journal of Econometrics*, **29**(3), 305–325. doi:[10.1016/0304-4076\(85\)90158-7](https://doi.org/10.1016/0304-4076(85)90158-7).
- McCullough BD (1997). “A Review of **RATS** v4.2: Benchmarking Numerical Accuracy.” *Journal of Applied Econometrics*, **12**(2), 181–190. doi:[10.1002/\(SICI\)1099-1255\(199703/04\)12:2<181::AID-JAE428>3.0.CO;2-V](https://doi.org/10.1002/(SICI)1099-1255(199703/04)12:2<181::AID-JAE428>3.0.CO;2-V).
- McCullough BD (1999). “Econometric Software Reliability: EViews, LIMDEP, SHAZAM and TSP.” *Journal of Applied Econometrics*, **14**(2), 191–202. doi:[10.1002/\(SICI\)1099-1255\(199903/04\)14:2<191::AID-JAE524>3.0.CO;2-K](https://doi.org/10.1002/(SICI)1099-1255(199903/04)14:2<191::AID-JAE524>3.0.CO;2-K).
- McCullough BD, Vinod HD (1999). “The Numerical Reliability of Econometric Software.” *Journal of Economic Literature*, **37**(2), 633–665. doi:[10.1257/jel.37.2.633](https://doi.org/10.1257/jel.37.2.633).
- Mersmann O (2024). **microbenchmark**: *Accurate Timing Functions*. R package version 1.5.0, URL <https://CRAN.R-project.org/package=microbenchmark>.
- Michelaki D, Tsagris M (2025). “An Overview of Economics and Econometrics Related R Packages.” *Stats*, **8**(4), 85. doi:[10.3390/stats8040085](https://doi.org/10.3390/stats8040085).
- ndarray Contributors (2024). **ndarray**: *An N-Dimensional Array for General Elements and for Numerics*. Rust crate version 0.16, URL <https://crates.io/crates/ndarray>.
- Ollama Inc (2024). *Ollama: Run Large Language Models Locally*. URL <https://ollama.ai>.
- Patil SG, Zhang T, Wang X, Gonzalez JE (2023). “Gorilla: Large Language Model Connected with Massive APIs.” *arXiv preprint arXiv:2305.15334*. URL <https://arxiv.org/abs/2305.15334>.
- Patil SG, Zhang T, Wang X, Gonzalez JE (2024). “Gorilla: Large Language Model Connected with Massive APIs.” In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Berkeley Function Calling Leaderboard, URL <https://gorilla.cs.berkeley.edu>.
- Pola-rs Contributors (2024). **polars**: *Blazingly Fast DataFrames in Rust and Python*. Version 0.46, URL <https://pola.rs/>.
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.

- Rust Foundation (2024). “The Rust Programming Language.” URL <https://www.rust-lang.org/>.
- rust-ml Contributors (2024). *linfa: A Rust Machine Learning Framework*. Rust crate, URL <https://github.com/rust-ml/linfa>.
- statrs Contributors (2024). *statrs: Statistical Computation Library for Rust*. Rust crate version 0.18, URL <https://crates.io/crates/statrs>.
- White H (1980). “A Heteroskedasticity-Consistent Covariance Matrix Estimator and a Direct Test for Heteroskedasticity.” *Econometrica*, **48**(4), 817–838. doi:10.2307/1912934.
- Zeileis A (2004). “Econometric Computing with HC and HAC Covariance Matrix Estimators.” *Journal of Statistical Software*, **11**(10), 1–17. doi:10.18637/jss.v011.i10.
- Zeileis A, Koenker R (2008). “Econometrics in R: Past, Present, and Future.” *Journal of Statistical Software*, **27**(1), 1–5. doi:10.18637/jss.v027.i01.

Affiliation:

First Author
 Department of Economics
 University of Somewhere
 Address Line
 City, Country
 E-mail: author@example.org
 URL: <https://example.org/>

Journal of Statistical Software

published by the Foundation for Open Access Statistics

MMMMMM YYYY, Volume VV, Issue II

doi:10.18637/jss.v000.i00

<http://www.jstatsoft.org/>

<http://www.foastat.org/>

Submitted: yyyy-mm-dd

Accepted: yyyy-mm-dd